

Алгоритм построения ДКА из регулярного выражения

Гафаров Т.Э. Б01-411

Что такое регулярные выражения, НКА и ДКА?

- *Регулярные выражения* - это способ описания паттернов в тексте. Они помогают найти определенные последовательности символов в строках. Например, с помощью регулярного выражения можно найти все вхождения слов в тексте или проверить, состоит ли строка только из цифр.
- *Недетерминированный конечный автомат (НКА)* - это модель, которая помогает проверять, подходит ли строка под регулярное выражение. В НКА из одного состояния можно переходить в несколько разных состояний по одному и тому же символу.
- *Детерминированный конечный автомат (ДКА)* - это улучшенная версия НКА, в которой для каждого состояния и символа есть только один переход. То есть, из каждого состояния по каждому символу возможен только один путь.
- ДКА проще и быстрее, чем НКА, потому что не нужно выбирать между несколькими возможными путями.

Проблема и постановка задачи

- *Проблема:* Простой перебор символов строки в поисках совпадений с регулярным выражением может работать слишком медленно, особенно когда регулярное выражение сложное.
- *Решение:* Для ускорения процесса используется ДКА, который позволяет быстро проверять соответствие строки регулярному выражению. Построение ДКА из регулярного выражения позволяет значительно ускорить обработку данных.

Проблема и постановка задачи

- *Проблема:* Простой перебор символов строки в поисках совпадений с регулярным выражением может работать слишком медленно, особенно когда регулярное выражение сложное.
- *Решение:* Для ускорения процесса используется ДКА, который позволяет быстро проверять соответствие строки регулярному выражению. Построение ДКА из регулярного выражения позволяет значительно ускорить обработку данных.

Постановка задачи:

- Разработать алгоритм, который будет преобразовывать регулярное выражение в детерминированный конечный автомат.

Математическая формулировка задачи

Что дано:

- Регулярное выражение - шаблон, описывающий набор допустимых строк
- По этому шаблону можно построить недетерминированный автомат (НКА), у которого из одного состояния может быть несколько путей по одному символу и есть пустые (ϵ) переходы.

Что требуется:

- Превратить НКА в детерминированный автомат (ДКА) - такой, где из каждого состояния по каждому символу ведёт не более одного перехода.
- Сократить ДКА до минимального - убрать лишние состояния, но так чтобы автомат по-прежнему распознавал в точности те же строки, что и исходное выражение.

Построение НКА по регулярному выражению

Дано: регулярное выражение

Задача: построить недетерминированный автомат, распознающий те же строки.

Метод: алгоритм Томпсона

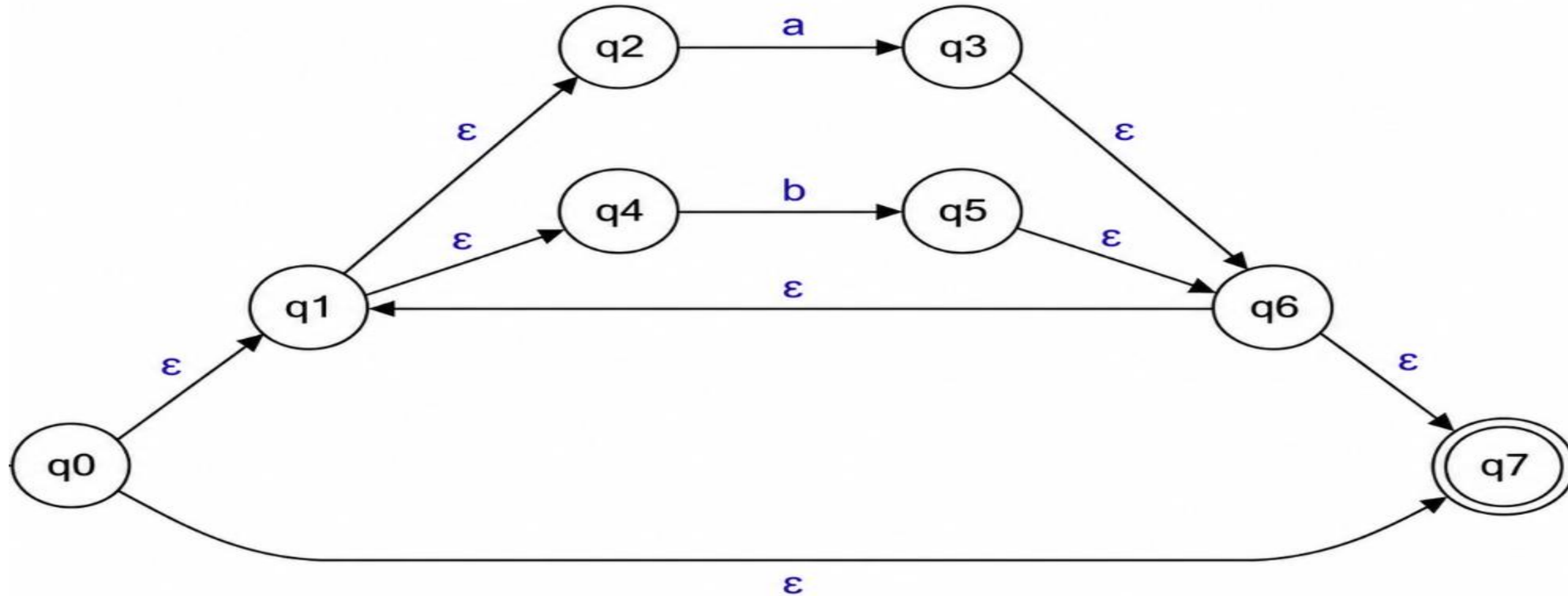
- Для каждого символа создаётся атомарный автомат (2 состояния)
- Операции $|$, $;$, $*$ объединяют автоматы ϵ -переходами
- Размер НКА пропорционален длине выражения ($\leq 2n$)

Результат: НКА с одним стартовым и одним финальным состоянием,
потенциально с несколькими переходами по одному символу.

Алгоритм Томпсона для построения НКА

- Строится рекурсивно по структуре выражения:
-
- 1) Базовые автоматы
 - – Для символа $c \in \Sigma$:
 - строится автомат из двух состояний $q_0 \rightarrow q_1$ с переходом по c .
 - q_0 – начальное, q_1 – конечное.
 - – Для пустого символа ε (используется редко):
 - автомат из одного состояния, которое одновременно начальное и конечное.
- 2) Составные операции
 - Пусть у нас уже есть НКА A для R_1 и НКА B для R_2 .
 - • Конкатенация $(R_1 \cdot R_2)$:
 - – Добавляем ε -переход из каждого финального состояния A
 - в начальное состояние B .
 - – Начало результирующего автомата = начало A .
 - – Финалы = финалы B .
 -
 - • Альтернатива $(R_1 \mid R_2)$:
 - – Создаём новое начальное состояние q_{start} .
 - – Проводим ε -переходы $q_{\text{start}} \rightarrow$ начало A и $q_{\text{start}} \rightarrow$ начало B .

Пример: НКА для $(a|b)^*$, построенный по Томпсону.



Алгоритм НКА в ДКА

Начало:

Берём начальное состояние НКА.

Находим все состояния, в которые можно попасть по ϵ -переходам (без чтения символов). Полученное множество становится первым состоянием будущего ДКА.

Помещаем это множество в очередь Q.

Шаг 1 (основной цикл):

Пока очередь Q не пуста:

- Извлекаем множество состояний q.
- Для каждого символа c из алфавита:
 - Из всех состояний внутри q делаем шаг по символу c.
 - Добавляем к результату все состояния, достижимые по ϵ .
 - Получаем новое множество состояний.
 - Если оно ещё не встречалось, добавляем его в очередь Q.
 - Запоминаем переход в ДКА: $q \xrightarrow{c} \text{новое множество}$.

Шаг 2:

Каждое множество из Q становится отдельным состоянием ДКА

Алгоритм детерминизации НКА в ДКА

Дано:

Σ – алфавит

Q – множество состояний НКА

s – начальное состояние НКА

T – множество конечных состояний НКА

δ – функция переходов НКА (может возвращать множество)

```
function getDFABynFA( $\{\Sigma, Q, s, T, \delta\}$ ):  
    start_set =  $\epsilon$ -closure( $\{s\}$ )           // Замыкание начального состояния  
    P.push(start_set)                       // Очередь для обработки  
    Qd =  $\emptyset$                            // Множество состояний ДКА (множества подмножеств)  
  
    while P  $\neq \emptyset$ :  
        pd = P.pop()                        // Извлекаем множество состояний НКА  
  
        for c  $\in \Sigma$ :                     // Для каждого символа алфавита  
            qd =  $\emptyset$   
            for p  $\in$  pd:                     // Для каждого состояния в pd  
                qd = qd  $\cup \delta(p, c)$       // Шаг по символу c  
            qd =  $\epsilon$ -closure(qd)           // Добавляем  $\epsilon$ -переходы  
  
            if qd  $\neq \emptyset$ :  
                 $\delta_d(pd, c) = qd$          // Записываем переход ДКА  
  
                if qd  $\notin$  Qd:               // Новое множество ещё не встречалось?  
                    P.push(qd)  
                    Qd.add(qd)  
  
    Td = {qd  $\in$  Qd |  $\exists p \in T: p \in$  qd}    // Финальные состояния ДКА  
    return ( $\Sigma, Qd, start\_set, Td, \delta_d$ )
```

Пример: преобразование НКА в ДКА

1. Инициализация

Стартовое множество: ϵ -замыкание($\{1\}$) = $\{1\}$ (ϵ -переходов нет)

Очередь $Q = [\{1\}]$

Состояния ДКА: пока нет

2. Обрабатываем $\{1\}$ (достаём из очереди, Q становится пустой)

• Символ a :

$$\delta(\{1\}, a) = \delta(1, a) = \{1, 2\}$$

$$\epsilon\text{-замыкание}(\{1, 2\}) = \{1, 2\}$$

Это новое множество $\rightarrow$ добавляем $\{1, 2\}$ в очередь и в ДКА.

Переход ДКА: $\{1\} \xrightarrow{a} \{1, 2\}$

• Символ b :

$$\delta(\{1\}, b) = \{1\}$$

$$\epsilon\text{-замыкание}(\{1\}) = \{1\}$$

Такое множество уже есть (это $\{1\}$), ничего не добавляем.

Переход ДКА: $\{1\} \xrightarrow{b} \{1\}$

Текущая очередь: $[\{1, 2\}]$

3. Обрабатываем $\{1, 2\}$ (достаём, очередь пуста)

• Символ a :

$$\delta(\{1, 2\}, a) = \delta(1, a) \cup \delta(2, a) = \{1, 2\} \cup \{2\} = \{1, 2\}$$

$$\epsilon\text{-замыкание}(\{1, 2\}) = \{1, 2\}$$

Множество уже известно, переход: $\{1, 2\} \xrightarrow{a} \{1, 2\}$

• Символ b :

$$\delta(\{1, 2\}, b) = \delta(1, b) \cup \delta(2, b) = \{1\} \cup \{2\} = \{1, 2\}$$

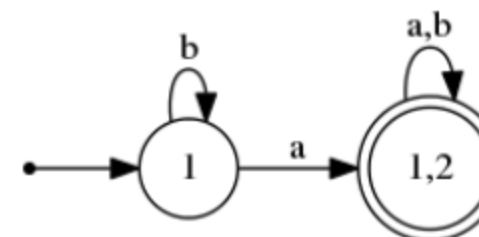
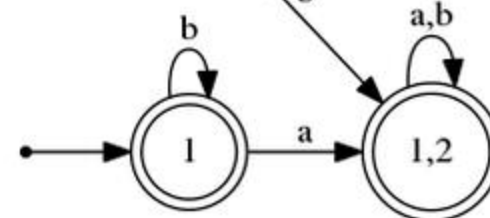
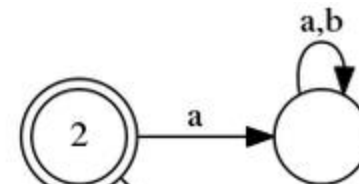
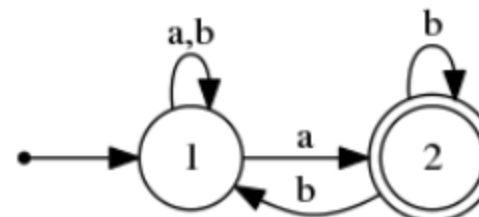
Переход: $\{1, 2\} \xrightarrow{b} \{1, 2\}$

Очередь пуста – построение закончено.

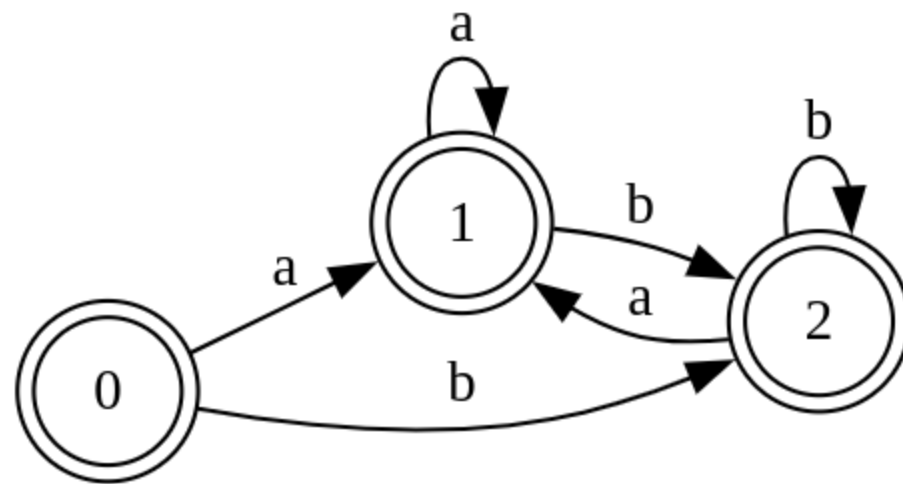
4. Финальные состояния ДКА

Финальным считается любое множество, содержащее конечное состояние НКА (2).

У нас такое множество: $\{1, 2\} \rightarrow$ оно и будет финальным.



Пример: НКА в ДКА для $(a|b)^*$



Минимизация ДКА

- ЦЕЛЬ: Уменьшить количество состояний ДКА, полностью сохранив принимаемый язык.
- Зачем это нужно?
 - Ускоряет обработку строк (меньше переходов)
 - Сокращает объём памяти, занимаемой автоматом

- Методы:

1. Метод таблицы эквивалентности

Итеративно помечает различимые пары состояний.

Время: $O(|\Sigma| \cdot n^2)$, Память: $O(n^2)$

2. Алгоритм Хопкрофта

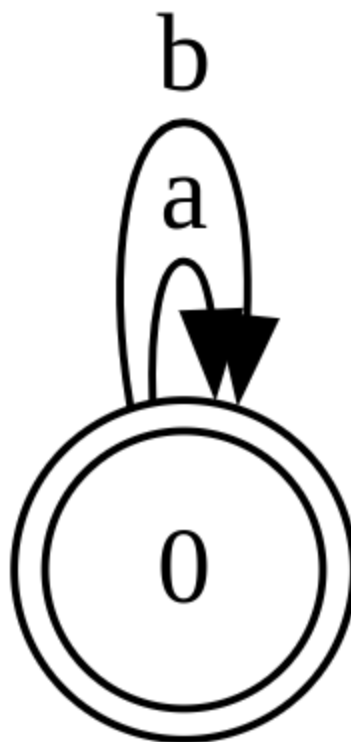
Разбивает состояния на классы эквивалентности за счёт обратных переходов.

Время: $O(|\Sigma| \cdot n \log n)$, Память: $O(|\Sigma| \cdot n)$

Минимизация ДКА

- • Цель: объединить неразличимые состояния, сохранив язык ДКА.
- • Критерий различимости: два состояния p, q различимы, если
 - $(p \in F) \oplus (q \in F)$ или $\exists c \in \Sigma: \delta(p, c)$ и $\delta(q, c)$ различимы.
 -
- Алгоритм:
 - 1. Построить таблицу $n \times n$, пометить пары $\{p, q\}$ с разной финальностью.
 - 2. Итеративно донашивать метки: если для непомеченной пары $\{u, v\}$
 - $\exists c$: пара $\{\delta(u, c), \delta(v, c)\}$ уже помечена $\rightarrow$ пометить $\{u, v\}$.
 - 3. Повторять, пока таблица не стабилизируется.
 - 4. Непомеченные пары образуют классы эквивалентности.
 - 5. Построить новый ДКА: состояния = классы, переходы по представителям.

Пример: минимизация для $(a|b)^*$



Оценка сложности

Этап	Время (худший случай)	Память
Регулярное в НКА	$O(k \cdot n)$, где n - длина выражения, k - размер алфавита	$O(n)$
НКА в ДКА	$O(2^s \cdot k)$, где s - число состояний НКА	$O(2^s)$ (размер результирующего ДКА)
Минимизация ДКА	$O(k \cdot m^2)$ $O(k \cdot m \log m)$	$O(m^2)$ $O(k \cdot m)$

Бенчмаркинг

Длина	NFA	DFA	Минимизация	Общее
10	0.01 мс	0.03 мс	0.02 мс	0.07 мс
100	0.07 мс	0.93 мс	0.29 мс	1.29 мс
1000	0.25 мс	71.9 мс	4.15 мс	76.3 мс
10000	18.1 мс	58.9 с	0.80 с	59.7 с

Полезные ссылки

- [Преобразование регулярного выражения в ДКА](#)
- [Алгоритм преобразования НКА в эквивалентный ДКА](#)
- [Конечные автоматы и регулярные выражения](#)
- [алгоритм Томпсона](#)
- [Минимизация ДКА, алгоритм за \$O\(n^2\)\$](#)
- [Минимизация ДКА, алгоритм Хопкрофта \(сложность \$O\(n \log n\)\$ \)](#)
- [Репозиторий с бенчмарками](#)